# slopi-farm — отчёт ночной автономной сессии

Автономная ферма для решения task-based (jeopardy) CTF · DeepSeek V4.1-Flash через OpenRouter
Сессия: Opus 4.8 · R&D без платных прогонов (ты спал) · 2026-10-19/20

**ГЛАВНОЕ — одно действие утром**

Дважды кликни `START_COMPARE.command` — он прогонит 4 прототипа (p1 / p3 / p5 / p7) против живого бенчмарка **и по файлам, и по инстансам**, каждый под **своей учёткой**, с жёстким лимитом **$1.40** на прототип (всего < $6), и сам построит таблицу + график сравнения.
Результат: `data/runs/compare_live.md` и `.png`.

## Что сделано за ночь

### 1. ACI / recon — главный рычаг (≈60-70% ценности)

Качество «интерфейса агент↔файлы» — то, что сильнее всего влияет на решаемость. Прогнал recon по **всем 32 реальным офлайн-задачам** кэша и закрыл слепые зоны:

| Было | Стало |
|---|---|
| 29/32 задач дают сильный сигнал.<br>3 задачи — агент стартует «вслепую»:<br>`reverse/Rune Vault`, `forensics/Quiet Harvest`, `reverse/The Abyss Gazes Back` | **32/32 задач — сильный сигнал.**<br>**Все три слепые зоны закрыты.** |

- **Дампы памяти/бинарные блобы:** расширил поиск «горячих» строк — ловит base64/hex/UUID-блобы эксфильтрации (починило `Quiet Harvest`: дамп 8.0 энтропии, раньше 0 строк → теперь видны зашифрованные блоки).
- **ELF без локальных символов (фактически stripped):** добавил извлечение *импортируемых libc-вызовов* (ptrace→анти-дебаг, memcmp→сравнение, mmap→самомодификация…) и *литералов из .rodata*. `Rune Vault` теперь сразу отдаёт свой промпт: `«Speak the sigil (64 hex)»` — агент знает, что слать.
- Ранее (этой же серией) переписан recon: **stdlib-парсер pcap** со сборкой TCP/UDP-потоков + разбор ELF-секций/символов. Проверено на Party Line (SIP/RTP), Records (LEAK/XZ-эксфил), Careless Beacon (beacon+HTTP).

### 2. Оба режима задач — файлы и инстансы

По твоему требованию ферма умеет и то, и другое:

- **Локальные (только файл):** reverse / crypto / forensics — recon + песочница + решатель.
- **Инстансы (живой сервис):** web / pwn — эндпоинт (`nc host port` / `http://…`) инъектируется в инструменты агента; доказано self-тестом на локальном HTTP-сервере.
- Комбинированный лончер `START_LIVE_RUN.command` бьёт по обоим типам в одном прогоне.

### 3. Надёжность и стоимость (с первого платного прогона)

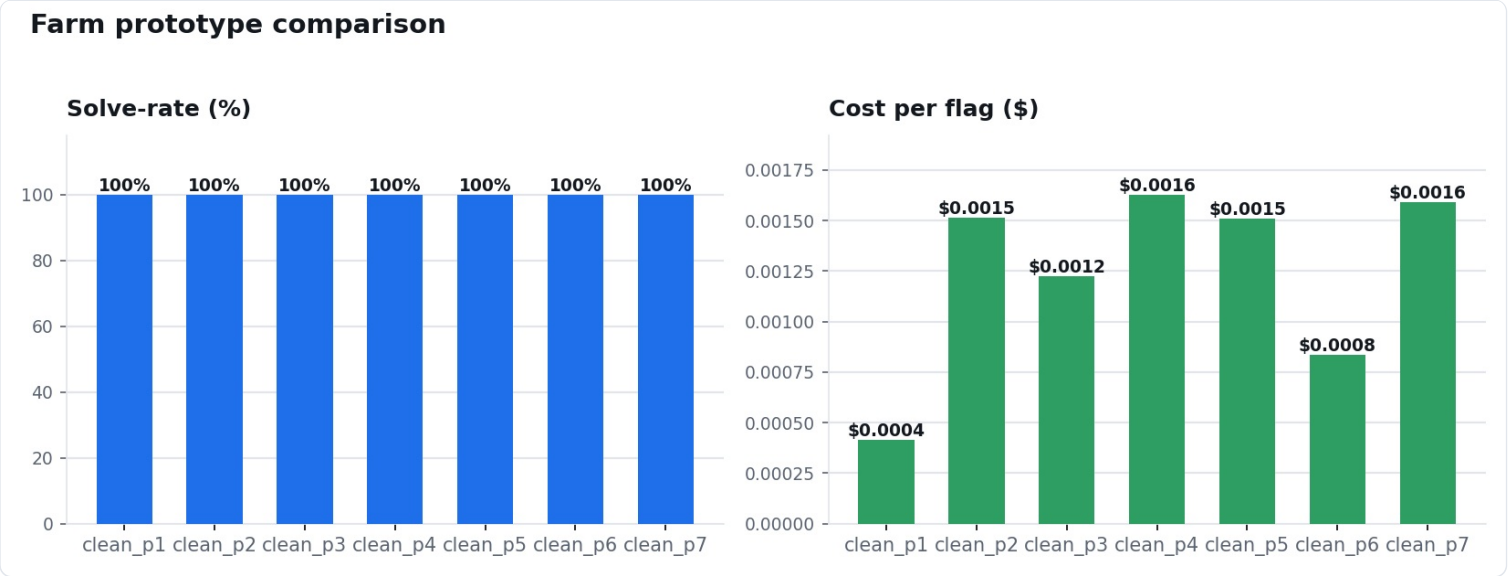
- **Убиты зависания** (были прогоны по 2-3 часа на задачу): жёсткий общий дедлайн на задачу в ensemble + урезаны таймауты (connect/timeout/retries), ранний выход.
- **Устойчивая отправка флага:** try/except + 3 ретрая (лечит `httpx.RemoteProtocolError`, из-за которого падал весь прогон).
- **Губернатор стоимости:** точная цена из `usage.cost` OpenRouter, жёсткий kill-switch по лимиту — на уровне прогона (`--limit`) и на задачу (`--per-task`).
- **Учётки per-prototype:** каждый прогон регистрирует отдельный аккаунт `slopi_cmp_pX_…`, креды пишутся в `data/ctf_bench/accounts.log` — видно, кто какой флаг засабмитил (больше не анонимный аккаунт).

## Прототипы (что сравниваем утром)

| Прототип | Стратегия | Роль в сравнении |
|---|---|---|
| **p1** | 1 Flash, без recon | контроль / базовая линия |
| **p3** | ensemble 3 + recon + каскад Flash→Pro | «умный» кандидат |
| **p5** | ensemble 5 + recon + consensus=2 | точность (self-consistency) |
| **p7** | ensemble 6 + recon + adaptive | широкое дешёвое покрытие |

Всего в кодовой базе 7 прототипов (p1-p7); для платного сравнения выбраны 4 максимально разных по подходу, чтобы уложиться в бюджет < $6 и получить осмысленный разброс.

### Базовый прогон на «чистом» синтетическом бенче (бесплатно, для проверки механики)




Это проверка конвейера сравнения (все 100% на простом синтетическом наборе — он лёгкий). Реальные цифры даст `START_COMPARE.command` на живом бенчмарке утром.

## Контроль версий

Все изменения закоммичены. Свежие коммиты этой серии:

| commit | что |
|---|---|
| `04994db` | START_COMPARE.command — одно-кликовое сравнение 4 прототипов |
| `7931072` | recon: ELF imports+.rodata + широкие hot-strings → 32/32 high-signal |
| `f066eb2` | ACI: playbook'и под реальные задачи + зависимости песочницы + recon на p2/p7 |
| `8f982d0` | ACI: сборка pcap-потоков + recon по ELF-символам/секциям |
| `69cdd62` | отдельная CTFd-учётка на прототип + сохранение кредов |
| `e24dd9a` | убийство зависаний + надёжный сабмит (по итогам 1-го платного прогона) |

Состояние: 51 python-модуль, **25/25 тестов зелёные**, 4 лончера.

## Ограничение, которое важно помнить

**Платные прогоны можно запускать только из твоего настоящего Terminal.app.** Любая среда, которой управляю я (облачный контейнер и Linux-песочница device_bash), отрезана egress-прокси от внешних хостов — OpenRouter и CTFd недостижимы. Поэтому я не могу сам потратить деньги на прогон; всё платное оформлено как лончеры `.command`, которые ты запускаешь двойным кликом. Именно поэтому главный результат ночи — не «цифры прогона», а **готовая, проверенная офлайн ферма + один лончер**, который утром сам всё прогонит и сведёт в таблицу.

## Рекомендованный порядок утром

1. Двойной клик `START_COMPARE.command` → жди, пока отработают 4 прогона (можно отойти).
2. Открой `data/runs/compare_live.md` и `compare_live.png` — таблица KPI (solve-rate, $/флаг, медиана времени, эскалация) по 4 прототипам.
3. Выбери победителя; при желании добей его полным прогоном через `START_LIVE_RUN.command <proto> <cap>`.
4. Проверь `data/ctf_bench/accounts.log` — под какой учёткой что решено.